FYS-STK3155 - Project 1 on Machine Learning
Regression analysis, resampling methods and gradient descent

Esme Hookham, Lukas Monrad-Krohn, and Andrea Rosendal
University of Oslo
(Dated: September 25, 2026)

In this project we study polynomial regression methods, resampling techniques and gradient-based optimisation using the Runge function. These methods are the foundation for more complex machine learning techniques used in large-language models and data science. Consequently, it is essential to understand their weaknesses, strengths and specific applications. Ordinary Least Squares (OLS), Ridge and Lasso regression are used to fit polynomial models of different complexities, while bootstrap and cross-validation are applied to evaluate model performance and investigate the bias-variance tradeoff. We further compare the closed-form method with the gradient descent to assess the tradeoff between accuracy and efficiency. Through these experiments we investigate how polynomial complexity, regularisation and optimisation methods affect prediction accuracy, convergence and computational cost. Our results show that [method] converge to the closed form in [n] iterations. The model [] with a [] polynomial degree and a learning rate [] provides the lowest error for this function. The results are used to compare the different regression methods and to relate their behaviour to the bias-variance tradeoff. [Adam] is the most versatile and accurate method due to the combination of the momentum and [], but [] and [] are well suited for [something else].

I. INTRODUCTION

For a mathematical regression model to be useful, it should be able to make sensible predictions for data it has not seen before, as well as describe the data it has been given. This increases in importance as the model becomes more complex. Increasing the polynomial degree of the model can allow it to perfectly reproduce the training data it has been given, but also lead to overfitting, where the model has become so excessively complex it is no longer useful and begins to fit noise. The Runge function [1]:

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1,1], \tag{1}$$

is a useful example of this problem since even though it is smooth and simple, high-degree polynomial approximations of it become unstable near the boundaries of the interval. The problem illustrates the bias-variance tradeoff: models that are too simple may have high bias and fail to capture the underlying function, while very flexible models have higher variance and become too sensitive to the particular training data [2]. The aim is therefore not simply to find a model that fits the training data as closely as possible, but to investigate how different choices of model complexity and regularisation affect generalisation to unseen data.

In this project, we study three linear regression methods: Ordinary Least Squares (OLS), Ridge regression and Lasso regression. First, we use OLS to fit increasingly flexible curves to the data. This gives us a simple baseline model and a direct mathematical formula to calculate the best parameters. Ridge regression is then introduced through an L_2 regularisation term, allowing us to observe how shrinking the regression coefficients affects the fitted model [3]. Lasso regression uses an L_1 penalty and provides a different form of regularisation which can drive individual coefficients to zero [4]. We construct the polynomial design matrices and implement the OLS and Ridge solutions ourselves, and then develop our own numerical implementation of Lasso using gradient-based optimisation.

We also investigate resampling methods for assessing model performance and model complexity. Bootstrap resampling [5] is used to generate multiple training sets and study the bias-variance tradeoff as the polynomial degree and number of observations are varied. We then use k-fold cross-validation [6, 7] to compare model performance across different polynomial degrees and regularisation strengths. The cross-validation procedure is implemented using the functionality provided by `scikit-learn` [8], while the interpretation of the resulting errors and the comparison with the bootstrap analysis are carried out by us. Particular attention is given to the treatment of preprocessing within each training fold, since information from the validation data should not influence the fitted model.

Finally, we investigate gradient-based optimisation methods as an alternative to the closed-form regression solutions. We implement gradient descent using both analytically derived gradients and gradients obtained through automatic differentiation, and use the closed-form OLS and Ridge solutions as reference solutions for testing convergence. We then extend the basic method with momentum [9], AdaGrad [10], RMSprop [11] and Adam [12], comparing their convergence behaviour and dependence on the learning rate. We additionally implement stochastic gradient descent, where the training data is divided into minibatches and the parameters are updated repeatedly over several epochs [13]. These optimisation methods are implemented by us, including the parameter-update rules and the experiments used to compare their convergence, accuracy and computational

cost. Automatic differentiation is provided through a library implementation [14], while the regression cost functions and optimisation procedures are constructed and tested independently.

The report is organised as follows: Section 2 introduces the models and methods. Section 3 provides a theoretical background, while our results are described and discussed in section 4. Lastly, a conclusion is presented in section 5. The code is available at `https://github.com/lukasMK01/FYS-STK4155/`.

II. CONCLUSION

In this paper, we have investigated the polynomial complexity, regularisation and optimisation methods using the Runge function. We find that the [] model with a [] polynomial degree, and a learning rate [] and lambda/penalty [] provides the lowest errors.

The bootstrap and the cross-validation produce minimal cost at a polynomial degree []. Above [] degrees, the variance increases.

Using various methods for the gradient descent approach, we find that some methods are more sensitive to the learning rate than others. Adam is a versatile method and is flexible with respect to the learning rate. RMSProp, however, does not converge towards the results of the OLS closed-form function (at $\lambda = 0$) for any learning rate.

We suggest using a [model] with [degree] and [other settings] for the Runge function. because... This model converges to the [OLS/Ridge] closed form in few iterations, and is flexible with regards to chosen learning rate and penalties. The closed form is the analytical result and therefore accurate result, but is highly computational, so unrealistic to use with large datasets. This setting is a balance between keeping low computational costs but maintaining predictive accuracy with the model.

The learning rates were scanned using only 16 different values but with a range of 1e-3 to 1, making the intervals between learning rates coarse. A finer interval between the learning rates could result in finding a convergence of the RMSprop. Another limitation is that we tested these methods on solely one function. Although we expect the results to be representative for most functions, we cannot be sure of that the suggestions for model and settings are applicable to other functions. Throughout the analysis, we needed to select specific values for various variables (lambda, learning rate, polynomial degree, etc) to use in further analysis. These values are chosen based on MSE and visual selection. Selecting other values will produce different results and possibly a different recommended model.

Further work can involve including a stopping function in the stochastic gradient descent, to reduce the computational cost by looping through all epochs when the method has converged. Also, applying this analysis to other functions can provide insight into how general the observed convergence behaviour is and how sensitive the different methods are to the choice of problem. We could also start measuring computational time specifically, not just number of iterations, since these two values are not always identical due to different levels of computational cost.

[1] C. Runge, Zeitschrift für Mathematik und Physik **46**, 224 (1901).

[2] M. Hjorth-Jensen, *Applied data analysis and machine learning: Lecture notes*, `https://github.com/CompPhysics/MachineLearning` (2026).

[3] A. E. Hoerl and R. W. Kennard, Technometrics **12**, 55 (1970).

[4] R. Tibshirani, Journal of the Royal Statistical Society: Series B **58**, 267 (1996).

[5] B. Efron, The Annals of Statistics **7**, 1 (1979).

[6] M. Stone, Journal of the Royal Statistical Society: Series B (Methodological) **36**, 111 (1974), ISSN 0035-9246.

[7] H. . U. S. Abedin, T; Xu, Scientific Reports **16** (2026).

[8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Journal of Machine Learning Research **12**, 2825 (2011).

[9] N. Qian, Neural Networks **12**, 145 (1999), ISSN 0893-6080.

[10] J. Duchi, E. Hazan, and Y. Singer, Journal of Machine Learning Research **12**, 2121 (2011).

[11] G. Hinton, *rmsprop: Divide the gradient by a running average of its recent magnitude* (2012), URL `https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf`.

[12] D. P. Kingma and J. Ba, in *International Conference on Learning Representations (ICLR)* (2015), arXiv:1412.6980.

[13] H. Robbins and S. Monro, The Annals of Mathematical Statistics **22**, 400 (1951).

[14] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al., *JAX: composable transformations of Python+NumPy programs*, `http://github.com/google/jax` (2018).

Appendix A: Use of AI/LLM tools

A.1 Tools used: Claude (claude.ai, September 2026); GitHub Copilot (VS Code, throughout); Writeful (included in overleaf).

A.2 Text writing and editing.

No AI tools were used for text writing and editing. *Writeful* AI paraphrasing suggestions are a default in overleaf, but we tried to ignore them. Claude AI has been used to debug latex code in one instance

`\usepackage[table]{xcolor}` creates errors in tables when no colouring is selected, 25.09.2026 13:41 CET).

A.3 Code generation and assistance.

Table I: AI use in coding (0=None, 1=Debugging, 2=Snippet, 3=Skeleton, 4=Substantial).

File & Code block	Level	Description
all	1	GitHub Copilot inline suggestions are a default setting and where occasionally accepted.
wip_project1.ipynb, task a-d)	1	discovered that a variable hadn't been updated